

ADIVINA QUIÉN — Documentación técnica

Diseño y Análisis de Algoritmos · Docente: López Juan Ignacio Alumna: Baptista Candela · Legajo: 1158810

1. Introducción

Este trabajo lo hice **sola**. La temática que elegí es una **academia de magia**: los 23 personajes son brujas y magos de la Academia Umbraluz, y la interfaz gráfica está ambientada en violeta y dorado siguiendo esa idea. Los atributos que pide el enunciado encajan sin forzar nada: el género distingue brujas de magos, la calvicie separa a los profesores rapados de los aprendices, y los tres colores de pelo son los que ya venían en la consigna.

Las estrategias que usé son estas. **Divide y Conquista** aparece en tres lugares: MergeSort para el ordenamiento inicial del lote de 23 personajes, búsqueda binaria para ubicar la posición de un alta individual posterior, y la reducción del conjunto de candidatos turno a turno, que es el corazón del juego. **Greedy** lo uso para la selección del filtro: la máquina elige qué preguntar con un criterio minimax, quedándose con la pregunta cuyo peor caso deja menos candidatos. Además uso **acceso por clave O(1)** con HashSet en dos lugares (los filtros ya preguntados y la verificación de que no haya personajes repetidos) y con HashMap para ir del id de un personaje a su tarjeta en el tablero.

Quiero aclarar dos cosas sobre esa lista, porque no todo está usado igual. Primero, **implementé también burbujeo e inserción simple, pero no se usan en el juego**: están únicamente para la tabla comparativa de tiempos de la sección 3. Segundo, además de la máquina greedy programé una **segunda máquina de búsqueda secuencial**, que juega peor a propósito: la hice para poder comparar las dos estrategias con números reales en el modo máquina contra máquina, no porque hiciera falta para que el juego funcione.

2. Justificación algorítmica de Divide y Conquista

2.1 Algoritmo de ordenamiento inicial

Los 23 personajes se declaran juntos y llegan agrupados solo por género, como pide el enunciado. Como tengo **el conjunto completo de entrada al mismo tiempo**, se lo puede partir a la mitad, que es justamente lo que MergeSort necesita.

```
public static List<Personaje> mergeSort(List<Personaje> lista,
                                       Comparator<Personaje> criterio) {
    if (lista.size() <= 1) {                // CasoBase(x)
        return new ArrayList<>(lista);      // SolucionDirecta(x)
    }
    int medio = lista.size() / 2;           // descomponer(x)
    List<Personaje> izq = mergeSort(lista.subList(0, medio), criterio);
    List<Personaje> der = mergeSort(lista.subList(medio, lista.size()), criterio);
    return mezclar(izq, der, criterio);      // combinar
}
```

Recurrencia: $T(n) = 2T(n/2) + \theta(n)$, porque el merge recorre los n elementos una vez. Es el caso de división con $a = 2$, $b = 2$, $k = 1$. Como $a = b^k$ ($2 = 2^1$), queda $\theta(n^k \log n) = \theta(n \log n)$.

Una aclaración honesta: el juego andaría igual sin ordenar nada. Con 23 personajes, recorrer la lista tal como viene funciona perfecto. Ordeno igual porque es la forma de aplicar la estrategia, y porque el orden habilita la búsqueda binaria del alta individual (sección 2.5), que sin lista ordenada no tendría sentido.

2.2 Elección entre MergeSort y QuickSort

	MergeSort	QuickSort
Complejidad promedio	$O(n \log n)$	$O(n \log n)$
Peor caso	$O(n \log n)$ garantizado	$O(n^2)$ si el pivot está mal elegido
Complejidad espacial	$O(n)$, arrays auxiliares	$O(\log n)$, ordena in-place
Estable	Sí	No

Elegí **MergeSort**, por dos motivos.

El primero es que Big O se evalúa sobre el peor caso, y MergeSort garantiza $\Theta(n \log n)$ sin importar cómo vengan los datos. QuickSort promedia lo mismo pero degrada a $\Theta(n^2)$ cuando el pivot cae en un extremo.

El segundo es más específico de mi caso, y es el que más me convenció. El peor caso de QuickSort se da con vectores ya parcialmente ordenados y mala elección de pivot. Y mis personajes llegan agrupados por género, o sea parcialmente ordenados. Elegir QuickSort acá sería elegir justo el algoritmo cuyo caso patológico coincide con mi entrada.

El punto débil de MergeSort, que sí tiene uno. Va creando listas nuevas en cada nivel de la recursión y guarda referencias a sublistas que después no vuelve a usar. Eso es complejidad espacial $O(n)$ contra el $O(\log n)$ de QuickSort, que solo usa la pila. Con 23 personajes es irrelevante, pero sería el factor limitante si el juego escalara a miles de personajes en un dispositivo con poca memoria, y ahí habría que reconsiderar la elección.

2.3 Criterio de ordenamiento

Ordeno por género — color de pelo — calvicie — anteojos, con un Comparator encadenado:

```
public static final Comparator<Personaje> POR_ATRIBUTOS =
    Comparator.comparing(Personaje::getGenero)
        .thenComparing(Personaje::getColorPelo)
        .thenComparing(Personaje::isCalvo)
        .thenComparing(Personaje::isUsaLentes);
```

Por qué por atributos y no por id. Si ordenara por id, la lista quedaría ordenada sola —los ids se asignan en orden de carga— y el ordenamiento no haría nada. Ordenar por atributos obliga a un reacomodamiento real.

Cómo va de la mano con la estructura de datos. El comparador define un orden total: como no hay dos personajes con la misma combinación de atributos, nunca devuelve 0 para dos distintos, así que cada uno tiene una posición única.

Eso es lo que me habilita a usar ArrayList con búsqueda binaria. Si el orden no fuera total, dos personajes competirían por la misma posición y la búsqueda quedaría ambigua. Y si usara LinkedList, el get (medio) costaría $O(n)$ en vez de $O(1)$ y el algoritmo entero perdería sentido: sería más lento que recorrer la lista de punta a punta. El criterio de orden, la estructura y el algoritmo se eligieron juntos.

2.4 Estrategia de selección del filtro

La máquina tiene que decidir qué preguntar primero. Acá entra Greedy, y el criterio es **minimax**: elige el filtro cuyo peor caso deje la menor cantidad de candidatos.

Esta es la evaluación real del primer turno sobre los 23 personajes, tal como la imprime el programa:

Pregunta	Cómo parte	Peor caso
¿Es un mago?	12 / 11	12
¿Tiene la cabeza rapada?	11 / 12	12
¿Usa anteojos?	11 / 12	12
¿Tiene el pelo colorado?	8 / 15	15
¿Tiene el pelo negro?	8 / 15	15
¿Tiene el pelo amarillo?	7 / 16	16

¿Por qué elige esa y no otra? Porque no sabe qué le van a contestar. Si eligiera pensando en que le va a ir bien, preguntaría “¿pelo amarillo?” esperando un sí que deja 7 candidatos; pero si le dicen que no le quedan 16, **peor que antes de preguntar**. Como no controla la respuesta, lo único que puede controlar es qué tan mal le puede ir.

¿Cambia la pregunta entre partidas? La primera no: con los 23 candidatos iniciales la evaluación siempre da lo mismo, así que arranca siempre con “¿Es un mago?”. A partir del segundo turno sí cambia, porque el conjunto de candidatos ya depende de la respuesta que dio el rival, y esa respuesta depende del personaje secreto, que se elige al azar. Dos partidas con secretos distintos toman caminos distintos desde el turno 2.

2.5 Función de evaluación del filtro

La métrica es Particion.peorCaso(), que devuelve Math.max(cumplen, noCumplen): el tamaño del grupo más grande. El **descarte garantizado** es total – peorCaso. En el turno 1, elegir “¿Es un mago?” descarta al menos 11 candidatos pase lo que pase.

Qué hace válido a ese filtro: que parte el conjunto lo más cerca posible del 50/50, que es lo que replica el descarte de la búsqueda binaria y da $\Theta(\log n)$ preguntas.

Empates. Pasa seguido: en el turno 1, tres filtros empatan en peor caso 12. **Me quedo con el primero de la lista**, no con uno al azar. Lo decidí así para que la máquina sea determinista: con el mismo personaje secreto siempre juega igual, y eso me permite reproducir una partida para mostrarla o compararla. Si desempatará al azar, cada corrida daría distinto y las mediciones de la sección 4 no serían comparables entre sí. El costo es que la elección arbitraria podría llevar a un camino más largo, cosa que discuto en la sección 4.3.

Cuando no quedan preguntas útiles. La función de factibilidad descarta los filtros ya usados y los que dejarían un lado vacío. Si la lista de factibles quedara vacía, la máquina arriesga con `candidatos.get(0)`, que es un **acceso por índice, $O(1)$** , lo más barato que hay. En mi catálogo esto nunca llega a pasar, porque los 23 personajes tienen combinaciones únicas y siempre existe una pregunta que separa. Pero el código lo contempla igual, porque con un catálogo con repetidos sí ocurriría.

2.6 Relación con el árbol de decisión

Una partida es un **recorrido desde la raíz hasta una hoja de un árbol binario de decisión**: los nodos internos son preguntas, las ramas son las respuestas sí/no, y las hojas son los personajes. Con 23 hojas, la altura mínima posible es $\lceil \log_2 23 \rceil = 5$.

Lo importante es que **el árbol no se construye entero en memoria**. Se recorre de forma perezosa: en cada nodo la máquina decide qué pregunta poner ahí mirando solo los candidatos que quedan.

Cómo piensa cada máquina. Programé dos, y la diferencia está justamente en cómo arman el árbol:

Máquina 1, greedy. En cada nodo evalúa los seis filtros contra los candidatos actuales y elige el que deja los grupos más parejos. Construye un árbol **equilibrado**, con altura cercana al mínimo de 5. Además aplica la función de factibilidad, así que nunca pone en el árbol una pregunta cuya respuesta ya está determinada.

Máquina 2, secuencial. Pregunta en un orden fijo declarado de antemano (primero los colores, después género, calvicie y anteojos), **sin evaluar cuánto corta cada filtro**. Es una búsqueda ingenua: arma el árbol que le toque. Como arranca por los colores, que son los filtros más desparejos, sus primeros niveles son desbalanceados y el árbol termina más alto.

Es importante aclarar que **la máquina 2 no juega al azar**. Su criterio se explica en una frase y es completamente reproducible: no hay ningún Random en su lógica de decisión. La hice así a propósito para que la comparación de la sección 4 mida la diferencia entre dos estrategias, y no entre una estrategia y el azar.

2.7 Cómo se descartan los candidatos en cada turno

Acá está el Divide y Conquista principal, en la clase Jugador:

El esquema dice	En mi juego es	Método
x	los personajes que todavía pueden ser	campo candidatos
CasoBase(x)	queda uno solo	esCasoBase()
SoluciónDirecta(x)	ese es el personaje	solucionDirecta()
descomponer(x)	separar en cumple / no cumple	descomponer()
combinar	seguir con el grupo de la respuesta real	recibirRespuesta()

El mecanismo concreto. Después de cada respuesta recorro la lista de candidatos una sola vez y armo dos listas nuevas: los que cumplen el filtro y los que no. Con la respuesta real me quedo con una y **descarto la otra entera**. No marco flags ni recorro después buscando los marcados: reemplazo la lista por la sublista que corresponde.

Costo: $\Theta(n)$, una pasada por los candidatos. Es lineal y no se puede bajar, porque hay que mirar a cada personaje al menos una vez para saber de qué lado cae.

Elegí crear listas nuevas en vez de eliminar elementos de la existente porque `ArrayList.remove()` desplaza todos los elementos posteriores, o sea $O(n)$ por eliminación, y en el peor turno habría que eliminar 16 de 23: quedaría $\Theta(n^2)$. Con una sola pasada queda $\Theta(n)$.

Una diferencia con el esquema genérico: el esquema resuelve *todos* los subproblemas y después combina. Acá resuelvo **uno solo**, porque la respuesta del rival me dice en cuál de los dos grupos está el personaje. Es la misma simplificación que hace la búsqueda binaria, y es lo que baja el orden de $\Theta(n)$ a $\Theta(\log n)$.

Dónde está la recursión. Esto fue lo que más me costó entender: no hay ningún método que se llame a sí mismo. La recursión está repartida **a lo largo de los turnos**, y cada turno resuelve el mismo problema con la mitad de los candidatos. Traza real de una partida:

```
descomponer(23) con "¿Es un mago?"      → [12 | 11]  respuesta SI → quedan 12
descomponer(12) con "¿Cabeza rapada?"   → [ 6 |  6]  respuesta SI → quedan  6
descomponer(6)  con "¿Usa anteojos?"    → [ 3 |  3]  respuesta NO → quedan  3
descomponer(3)  con "¿Pelo colorado?"    → [ 1 |  2]  respuesta NO → quedan  2
descomponer(2)  con "¿Pelo negro?"      → [ 1 |  1]  respuesta SI → queda  1
CasoBase → SoluciónDirecta
```

23 – 12 – 6 – 3 – 2 – 1, la misma forma que el $100 - 50 - 25 - 12 - 6 - 3 - 1$ del ejemplo de clase. Para que no quedara solo como interpretación mía, escribí además la clase `BuscadorRecurso`, donde el mismo algoritmo está con la recursión explícita: ahí `resolver()` sí se llama a sí mismo.

2.8 Criterio de parada y suposición final

Son dos cosas distintas y las separo.

(a) Fin del turno de la máquina. La función de corte es `esCasoBase()`. Cada vez que le toca jugar, la máquina se pregunta si le queda un solo candidato. Si le quedan dos o más, elige un filtro y pregunta: ahí termina su turno, porque la respuesta no depende de ella. Si le queda uno solo, no pregunta: pasa a la suposición. O sea que el turno termina siempre después de **una** acción, pregunta o suposición, nunca las dos.

(b) Fin del juego. La partida termina cuando alguien arriesga un nombre. Si acierta gana; **si se equivoca pierde la partida**. Elegí esa regla, que es la clásica del juego, porque es la que le da sentido a la decisión de arriesgar: si equivocarse no costara nada, la estrategia óptima sería adivinar en el turno 1 con probabilidad $1/23$ e ir descartando.

Con esa regla, `MaquinaGreedy` **solo arriesga cuando le queda un único candidato**, o sea cuando la probabilidad de acertar es 1. No arriesga antes por más tentador que sea, porque el costo de fallar es perder.

Y acá hay algo que quiero destacar, porque conecta con el diseño del catálogo: **cundo la máquina llega al caso base, el candidato que queda ES el personaje secreto**. No es una apuesta. Eso funciona porque los 23 personajes tienen combinaciones de atributos únicas (sección 2.10). Si hubiera repetidos, el caso base dejaría dos o tres empatados y la suposición final sería una moneda al aire.

2.9 Manejo de la dependencia lógica

Los seis filtros **no son todos independientes**. Género, calvicie y anteojos son binarios e independientes entre sí, pero los tres de color están ligados: si un personaje tiene el pelo colorado, no lo tiene negro ni amarillo.

La consecuencia práctica: después de responder “¿colorado? NO” y “¿negro? NO”, el filtro “¿amarillo?” **ya está determinado**, la respuesta es forzosamente sí para todos los candidatos que quedan. Preguntarlo gasta un turno sin descartar a nadie.

Lo resuelve la función de factibilidad:

```
if (aplicaFactibilidad() && descomponer(candidatos, f).esInutil()) continue;
```

No lo resolví con reglas escritas a mano del estilo “si ya preguntaste dos colores no preguntes el tercero”. Se resuelve **midiendo la partición real** sobre los candidatos actuales, así que funciona para cualquier dependencia entre atributos, no solo para la de los colores. Si mañana agregó un atributo correlacionado con otro, el mecanismo lo detecta solo.

En cuanto al **acoplamiento**, esta dependencia lógica no se filtra al diseño de clases: los seis filtros son subclases hermanas de `Filtro` y ninguna conoce a las otras. La relación entre ellos aparece solo en tiempo de ejecución, al medir las particiones. En la sección 4.2 se mide cuánto aporta esta función, y resulta ser el elemento del `greedy` con más impacto.

2.10 Precondiciones y validaciones

Validación	Dónde	Qué previene
Los 23 personajes tienen combinaciones únicas	<code>VerificadorCatalogo.verificarUnicidad()</code>	Que la máquina quede con candidatos empatados e imposibles de separar
El catálogo tiene exactamente 23	ídem	Que se rompa el requisito del enunciado sin que nadie lo note
<code>MergeSort</code> y la inserción binaria dan la misma lista	<code>verificarQueCoincidan()</code>	Que uno de los dos algoritmos esté mal implementado

Validación	Dónde	Qué previene
Un jugador no puede tener cero candidatos	constructor de Jugador	Estados imposibles
El personaje secreto no puede ser nulo	constructor de OraculoPersonaje	Un NullPointerException en medio de la partida
No se puede revelar el secreto con la partida en curso	Partida.getSecretoDe()	Que una vista haga trampa mostrando el personaje del rival
No se puede jugar una partida terminada	Partida.aplicarJugada()	Estados inconsistentes
La opción del menú está en rango	JuegoConsola.leerEntero()	Que el programa corte por una entrada inválida

Las tres primeras **se ejecutan al arrancar** y lanzan excepción si fallan. Prefiero que el programa reviente al iniciar antes que enterarme del problema en medio de la defensa.

Los pensamientos de la máquina, logueados

Esta es la parte que me parece más importante de esta sección. La máquina no decide en silencio: **imprime su razonamiento completo, turno por turno, en la consola y también en el panel de la interfaz gráfica**. Salida real:

```
--- TURNO 2 - juega MAQUINA (23 candidatos) ---
[MAQUINA] evaluo 6 filtros sobre 23 candidatos:
-> ¿Es un mago?           parte 12/11  peor caso: 12
    ¿Tiene la cabeza rapada? parte 11/12  peor caso: 12
    ¿Usa anteojos?        parte 11/12  peor caso: 12
    ¿Tiene el pelo colorado? parte 8/15   peor caso: 15
    ¿Tiene el pelo negro?  parte 8/15   peor caso: 15
    ¿Tiene el pelo amarillo? parte 7/16   peor caso: 16
[MAQUINA] SELECCION greedy: "¿Es un mago?" porque su peor caso (12) es el
menor. Descarta al menos 11 candidatos pase lo que pase.
[MAQUINA] respuesta SI -> combinar: sigo con "cumplen". 23 -> 12 (descarte 48%)
```

Se puede leer línea por línea qué patrón está actuando: las primeras seis líneas son **greedy evaluando** (función de selección), la línea de SELECCION es **la decisión tomada**, y la última es **Divide y Conquista combinando**. Esa correspondencia es lo que permite verificar que el código coincide con las estrategias vistas en clase sin tener que leer el código.

Técnicamente esto funciona porque el motor **nunca llama a System.out**: le pasa los mensajes a una interfaz RegistroRazonamiento, con tres implementaciones (consola, Swing y silenciosa para las simulaciones masivas). Por eso el mismo log aparece idéntico en las dos vistas sin duplicar una línea de código.

3. Tabla comparativa de tiempos

Medido con System.nanoTime() sobre **la misma lista de 23 personajes**, promediando 20.000 repeticiones por ejecución con calentamiento previo de la JVM. La clase es BenchmarkOrdenamiento.

Algoritmo	Complejidad	Ejec. 1	Ejec. 2	Ejec. 3	Promedio
MergeSort	$\Theta(n \log n)$	0,0075 ms	0,0061 ms	0,0057 ms	0,0064 ms
Burbujeo	$O(n^2)$	0,0057 ms	0,0054 ms	0,0054 ms	0,0055 ms
Inserción simple	$O(n^2)$	0,0032 ms	0,0022 ms	0,0022 ms	0,0025 ms

3.1 ¿Es significativa la diferencia para $n = 23$?

No, y hay que decirlo con todas las letras: no solo no es significativa, sino que MergeSort quedó último. La inserción simple resultó ser la más rápida de las tres: 0,0025 ms contra 0,0064 ms de MergeSort, o sea que el algoritmo cuadrático fue **dos veces y media más rápido** que el $n \log n$.

Hay dos motivos, y los dos son reales:

1. Con n chico las constantes pesan más que el orden. MergeSort reserva memoria para las sublistas en cada nivel de la recursión. Ese costo fijo no se amortiza con 23 elementos.

2. Mis personajes llegan agrupados por género, o sea parcialmente ordenados. Esa es la **mejor entrada posible** para inserción simple, que en su mejor caso es $O(n)$.

Para comprobar que era eso y no otra cosa, medí lo mismo con **la misma lista pero desordenada al azar**, y la relación se dio vuelta:

Algoritmo	Tal como llegan	Desordenados
MergeSort	0,0064 ms	0,0055 ms
Burbujeo	0,0055 ms	0,0082 ms
Inserción simple	0,0025 ms	0,0047 ms

MergeSort **mejoró** al desordenar la entrada y los dos cuadráticos empeoraron. Eso confirma que la ventaja de los cuadráticos no venía del algoritmo sino del orden en que llegaban los datos. MergeSort tarda casi lo mismo en los dos casos, que es justamente lo que significa tener el peor caso garantizado.

Vale la pena mirar también la dispersión entre ejecuciones: **el ruido de medición es del mismo orden que la diferencia entre algoritmos**. Cualquier conclusión sacada de comparar dos promedios que difieren en microsegundos no sería confiable.

3.2 Pero eso no valida la elección

Que a $n = 23$ dé lo mismo no significa que dé lo mismo elegir. Big O evalúa el **peor caso** y sirve para saber si el diseño escala. Estas son mediciones reales del mismo programa con listas más grandes:

n	MergeSort	Burbujeo	Inserción	Burbujeo / MergeSort
23	0,0048 ms	0,0066 ms	0,0036 ms	1,4×
100	0,032 ms	0,184 ms	0,059 ms	5,8×
500	0,214 ms	4,88 ms	1,58 ms	22,9×
2.000	1,06 ms	79,7 ms	23,97 ms	75,2×
5.000	2,98 ms	498,6 ms	157,4 ms	167,2×

Y la proyección teórica en cantidad de comparaciones:

n	$n \log_2 n$ aprox.	n^2	Factor
23	~104	529	~5×
100	~664	10.000	~15×
1.000	~9.966	1.000.000	~100×
10.000	~132.877	100.000.000	~750×

Los tiempos medidos siguen la forma de la proyección: el factor crece con n de 1,4× a 167×, o sea dos órdenes de magnitud en el rango medido.

Una aclaración sobre por qué los factores medidos quedan por debajo de los teóricos (a $n = 5.000$ la proyección pura daría alrededor de 400× y yo medí 167×). La proyección cuenta **solo comparaciones**, y el tiempo real incluye además la memoria que reserva cada algoritmo. MergeSort crea listas nuevas en cada nivel de la recursión y burbujeo no reserva nada, así que MergeSort arrastra un costo constante que la proyección teórica no ve. Eso achica la brecha real sin cambiar la tendencia: la diferencia sigue creciendo con n , que es lo que importa.

La conclusión, entonces, es doble. Para 23 personajes elegir uno u otro no cambia nada perceptible, y si el criterio fuera solo la velocidad medida hoy, la inserción simple sería la elección. Elijo MergeSort igual porque su $\Theta(n \log n)$ es una **garantía, no un promedio**: los cuadráticos son rápidos solo si tienen suerte con la entrada, y acá la tuvieron porque los datos vienen medio ordenados. Con 1.000 personajes la diferencia pasa de imperceptible a molesta, y con 10.000 a inutilizable.

4. Justificación de la estrategia Greedy

4.1 Los cinco elementos

Elemento	En mi juego	Dónde está
Conjunto de candidatos	los filtros que todavía no pregunté	<code>filtrosFactibles()</code>
Función de selección	el filtro de menor peor caso	<code>MaquinaGreedy.elegirFiltro()</code>
Función de factibilidad	que no esté usado y que separe en dos grupos no vacíos	<code>filtrosFactibles()</code>
Función de solución	queda un solo personaje	<code>esCasoBase()</code>
Objetivo	minimizar la cantidad de preguntas	

Por qué elegí esta variante. Comparé mentalmente dos criterios posibles: elegir el filtro que más descarta en el mejor caso, o el que menos deja en el peor. Me quedé con el segundo (minimax) porque la máquina no controla la respuesta. Está desarrollado en la sección 2.4 con la tabla de los seis filtros.

Cómo calculo el descarte. Para cada filtro cuento cuántos candidatos lo cumplen y cuántos no, y me quedo con el que deja los dos grupos más parejos, o sea el más cercano al 50/50. `peorCaso()` devuelve `Math.max(cumplen, noCumplen)` y el descarte garantizado es `total - peorCaso`.

Por qué es eficiente. Es exactamente el mismo razonamiento que la búsqueda binaria: cada pregunta bien elegida descarta aproximadamente la mitad, así que con 23 personajes hacen falta unas 5 preguntas ($\log_2 23 \approx 4,5$). Una estrategia lineal del tipo “¿sos Aurelia?”, “¿sos Beatrix?” necesitaría hasta 23 intentos. La evaluación cuesta $\Theta(f \cdot n)$ por turno, con $f = 6$ filtros y $n \leq 23$: a lo sumo 138 comparaciones, que es nada a cambio de pasar de 23 turnos a 6.

4.2 Medición: qué aporta cada elemento del greedy

Como no quería afirmar “greedy es mejor” sin poder demostrarlo, programé `SimulacionEstrategias`, que juega **las 23 partidas posibles** (una por cada personaje secreto) con tres estrategias sobre el mismo motor.

Estrategia	Peor caso	Promedio	Aciertos
Greedy completo (selección + factibilidad)	6 turnos	5,61	23/23
Secuencial con factibilidad	6 turnos	5,61	23/23
Secuencial pura (sin greedy)	7 turnos	6,96	23/23

Separando el aporte de cada elemento:

Elemento	Cuánto mejora el promedio
Función de factibilidad sola	-1,35 turnos
Función de selección minimax	0,00 turnos

El resultado no fue el que esperaba. En mi juego, lo que realmente mejora el rendimiento no es elegir la mejor pregunta, sino descartar las preguntas cuya respuesta ya está determinada. Se ve así en el log:

```
--- TURNO 6 - juega SECUENCIAL (8 candidatos) ---
[SECUENCIAL] SELECCION secuencial: "¿Tiene el pelo amarillo?", el primero
              pendiente del orden fijo. No evaluo cuanto corta.
[SECUENCIAL] respuesta NO → Candidatos 8 → 8 (descarte 0%)
```

Gastó un turno entero sin descartar a nadie.

¿Por qué la selección minimax no aporta nada? Porque **mi catálogo es uniforme**: como usé una combinación de cada, los tres filtros binarios cortan 12/11 o 11/12. Están empatados, y greedy no puede sacar ventaja cuando no hay una mejor opción para descubrir. La mantengo igual porque garantiza que nunca se elija un filtro malo (los de color cortan 8/15 y 7/16) y porque con un catálogo desbalanceado sí haría diferencia.

Lo que la medición me mostró es que **la ventaja de un algoritmo voraz depende de cómo estén armados los datos, no solo del algoritmo**.

4.3 ¿Existe un escenario donde mi Greedy no sea óptimo?

Para mi catálogo concreto, no, y se puede demostrar con un argumento de información:

Cada pregunta de sí/no aporta como máximo 1 bit. Para distinguir entre 23 personajes hacen falta al menos $\lceil \log_2 23 \rceil = 5$ preguntas, haga lo que haga cualquier algoritmo. Mi greedy resuelve en 5 preguntas en el peor caso. Como el mínimo teórico y el resultado coinciden, **ninguna estrategia puede hacerlo mejor**.

Pero eso vale para este catálogo, no en general, y es exactamente la lección del problema del cambio de monedas visto en clase: con el sistema decimal la estrategia voraz da el óptimo, y con el sistema británico previo a 1971 el **mismo algoritmo** da 4 monedas cuando el óptimo son 3. No cambió el algoritmo, cambiaron los datos de entrada.

El análogo en mi juego son tres escenarios concretos:

Atributos correlacionados. Si dos atributos estuvieran casi perfectamente correlacionados (por ejemplo “usa sombrero” y “tiene el pelo tapado”), la pregunta que más descarta ahora podría dejar un subconjunto imposible de partir bien después. Greedy no mira el futuro: elegiría el descarte inmediato y perdería la secuencia más corta. Un algoritmo con lookahead, o el árbol de decisión óptimo construido con programación dinámica, encontraría el camino mejor.

Distribución desbalanceada. Si un atributo lo tuvieran 22 de 23 personajes, ninguna pregunta partiría cerca del 50/50 y el peor caso se degradaría hacia $O(n)$, acercándose a la fuerza bruta.

Los empates. Como describí en 2.5, cuando varios filtros empatan me quedo con el primero. Esa elección arbitraria podría llevar a un camino más largo, y greedy nunca reconsidera una decisión ya tomada. En mi catálogo no pasa porque todos los caminos resultan equivalentes, pero es una debilidad estructural del enfoque, no un detalle de implementación.

En resumen: **mi greedy es óptimo porque mi catálogo es uniforme y sus atributos son independientes**, no porque el algoritmo sea óptimo por naturaleza.

5. Algoritmos vistos en clase que NO utilicé

Algoritmo	Por qué no aplica
QuickSort	Su peor caso $\Theta(n^2)$ se da con vectores parcialmente ordenados y mala elección de pivot, que es exactamente cómo llegan mis 23 personajes (agrupados por género). Elegirlo sería elegir el algoritmo cuyo caso patológico coincide con mi entrada. Desarrollado en 2.2.
Dijkstra	Requiere un grafo dirigido con costos y un vértice de origen. Mi juego no modela distancias ni costos entre personajes: el descarte es por atributos booleanos, no por peso de aristas. No hay ningún camino que minimizar.
Prim / Kruskal	Resuelven árboles de recubrimiento mínimo sobre grafos con costos. No existe el problema de conectar todos los personajes al mínimo costo: los personajes no están relacionados entre sí por aristas, son elementos independientes de un conjunto.
Código de Huffman	Es compresión por frecuencia de símbolos. En mi juego no hay codificación ni compresión de datos. Comparte con mi solución la idea del árbol binario, pero Huffman lo construye de abajo hacia arriba combinando frecuencias, y acá el árbol se recorre de arriba hacia abajo eligiendo preguntas.
Fibonacci recursivo / recursión de bifurcación	$\Theta(2^n)$ sin ninguna ventaja. En mi problema no hay un subproblema que se divida en dos ramas que haya que resolver las dos: la respuesta del rival me dice cuál rama seguir y la otra se descarta entera.
Programación dinámica	Sirve cuando los subproblemas se repiten , como el Fibonacci ingenuo que recalcula lo mismo millones de veces. Acá cada respuesta parte el conjunto en dos subconjuntos disjuntos : nunca vuelvo a visitar el mismo subconjunto, así que no hay nada que memorizar. La menciono como mejora futura para construir el árbol de decisión óptimo.

Algoritmo	Por qué no aplica
Búsqueda exhaustiva del mejor orden de preguntas	Serían $6! = 720$ órdenes posibles, computacionalmente tratable. No aporta porque greedy ya alcanza la cota inferior de 5 preguntas (sección 4.3): no hay una secuencia mejor que encontrar.
Divide y Conquista para elegir el filtro	Lo evalué: partir la lista de filtros a la mitad, buscar el mínimo en cada mitad y combinar. Lo descarté porque encontrar el mínimo de una lista obliga a mirar todos sus elementos igual, así que D&C no baja el orden (sigue siendo $\Theta(f)$) y solo agrega llamadas a la pila.
Burbujeo e inserción simple	Los implementé, pero no se usan en el juego : están solo para la tabla comparativa de tiempos de la sección 3.

6. Notación Big O para la aplicación

6.1 Función por función

Uso dos variables: **n** = cantidad de personajes (23) y **f** = cantidad de filtros (6). Las explico por separado en vez de forzar todo a una sola n, porque son magnitudes independientes: podría agregar filtros sin agregar personajes.

Función / método	Temporal	Espacial	Justificación
OrdenadorPersonajes.mergeSort()	$\Theta(n \log n)$	$\Theta(n)$	D&C: $T(n) = 2T(n/2) + \theta(n)$, $a=2$, $b=2$, $k=1$, $a=b^k$
OrdenadorPersonajes.burbujeo()	$O(n^2)$	$\Theta(n)$	Dos ciclos anidados. No se usa en el juego
CatalogoPersonajes.buscarPosicion()	$\Theta(\log n)$	$\Theta(1)$	Búsqueda binaria: $T(n) = T(n/2) + c$, $a=1$, $b=2$, $k=0$
CatalogoPersonajes.insertarOrdenado()	$\Theta(n)$	$\Theta(1)$	$\Theta(\log n)$ para ubicar + $\Theta(n)$ por el desplazamiento
VerificadorCatalogo.verificarUnicidad()	$\Theta(n)$	$\Theta(n)$	Una pasada con HashSet, $add()$ en $O(1)$
Jugador.descomponer()	$\Theta(n)$	$\Theta(n)$	Un recorrido de la lista de candidatos
Jugador.recibirRespuesta()	$\Theta(n)$	$\Theta(n)$	Llama a descomponer y reemplaza la lista
MaquinaGreedy.elegirFiltro()	$\Theta(f \cdot n)$	$\Theta(f)$	Ciclo anidado: por cada filtro, recorre los candidatos
JugadorMaquina.filtrosFactibles()	$\Theta(f \cdot n)$	$\Theta(f)$	Ídem: evalúa la partición de cada filtro
Partida.aplicarJugada()	$\Theta(n)$	$\Theta(1)$	Delega en recibirRespuesta
TableroPersonajes.pintar()	$\Theta(n)$	$\Theta(n)$	HashSet de ids vivos + una pasada por las tarjetas
CatalogoPersonajes.buscarPorNombre()	$O(n)$	$\Theta(1)$	Búsqueda lineal. Solo se usa al elegir personaje
Una partida completa	$\Theta(f \cdot n)$	$\Theta(n)$	Ver abajo
La aplicación	$\Theta(n \log n)$	$\Theta(n)$	Dominada por el ordenamiento inicial

Las dos funciones con **ciclos anidados** son `elegirFiltro()` y `filtrosFactibles()`: un `for` sobre los filtros y adentro un recorrido de los candidatos. Son la parte más pesada de la capa de servicios, y quedan en $\Theta(f \cdot n) = 138$ operaciones por turno como máximo.

6.2 El costo de una partida completa

Uno esperaría multiplicar el costo por turno por la cantidad de turnos: $\Theta(f \cdot n \cdot \log n)$. Pero el conjunto se achica a la mitad en cada turno, así que la suma real es una serie geométrica:

$$f \cdot (n + n/2 + n/4 + \dots + 1) = f \cdot 2n \rightarrow \Theta(f \cdot n)$$

El primer turno domina el costo de todos los demás juntos. Me pareció el resultado más lindo del análisis, porque es contraintuitivo: la partida entera cuesta lo mismo, en orden, que dos primeros turnos.

6.3Cuál es la notación más certera para la app

$\Theta(n \log n)$, dominada por el ordenamiento inicial del catálogo.

Aplicando las reglas de simplificación: el ordenamiento aporta $\Theta(n \log n)$ y la partida aporta $\Theta(f \cdot n)$. Como f es una constante que no depende de n , el segundo término es $\Theta(n)$, y $n \log n$ crece más rápido que n . Descartando el término de menor crecimiento queda $\Theta(n \log n)$.

Qué significa eso en el contexto de este juego. Que lo más caro que hace el programa es acomodar los personajes **una sola vez al arrancar**, y que jugar es más barato que prepararse para jugar. Con 23 personajes el arranque es instantáneo.

Si escalara a **1.000 personajes**: el ordenamiento pasaría de ~ 104 a ~ 9.966 comparaciones, o sea unos 3 ms. La partida costaría $\Theta(f \cdot n) = 6.000$ operaciones por partida, también despreciable, y la cantidad de preguntas subiría solo de 5 a 10, porque crece logarítmicamente. **El juego seguiría siendo jugable**, y eso es precisamente lo que valida el diseño: si hubiera usado burbujeo para ordenar y fuerza bruta para preguntar, con 1.000 personajes el arranque tomaría 1.000.000 de comparaciones y una partida necesitaría hasta 1.000 preguntas.

Hay una salvedad que quiero dejar anotada: con 1.000 personajes **el catálogo ya no podría tener combinaciones únicas**, porque mis 4 atributos solo generan 24 combinaciones distintas. Habría que agregar atributos, y ahí f dejaría de ser una constante chica. Ese sería el verdadero límite del diseño actual, y no el ordenamiento.